Extend Java EE containers with cloud characteristics

Strategies and patterns to extend JEE containers/apps with parallelism, elasticity, multi-tenancy, and security

Skill Level: Advanced

Jayakrishnan Ramdas (jkramdas@infosys.com)
Senior Technology Architect
Infosys LTD

J. Srinivas (jsrinivas@infosys.com)
Principal Architect
Infosys LTD

16 May 2011

In this article, the authors outline the basic characteristics of cloud applications and Java™ Enterprise Edition applications, compare their similarities and contrast their differences, and then define a set of strategies and provide patterns to extend Java EE container and application with such cloud characteristics as parallelism, elasticity, multi-tenancy, and security.

The Java Enterprise Edition (JEE) architecture is based on components with features that effectively manage application transactions and statefulness, multi-threading, and resource pooling. A JEE application is easier to write even with complex requirements since the business logic is organized in reusable components and the server provides the underlying services — in the form of a *container* — for each component type.

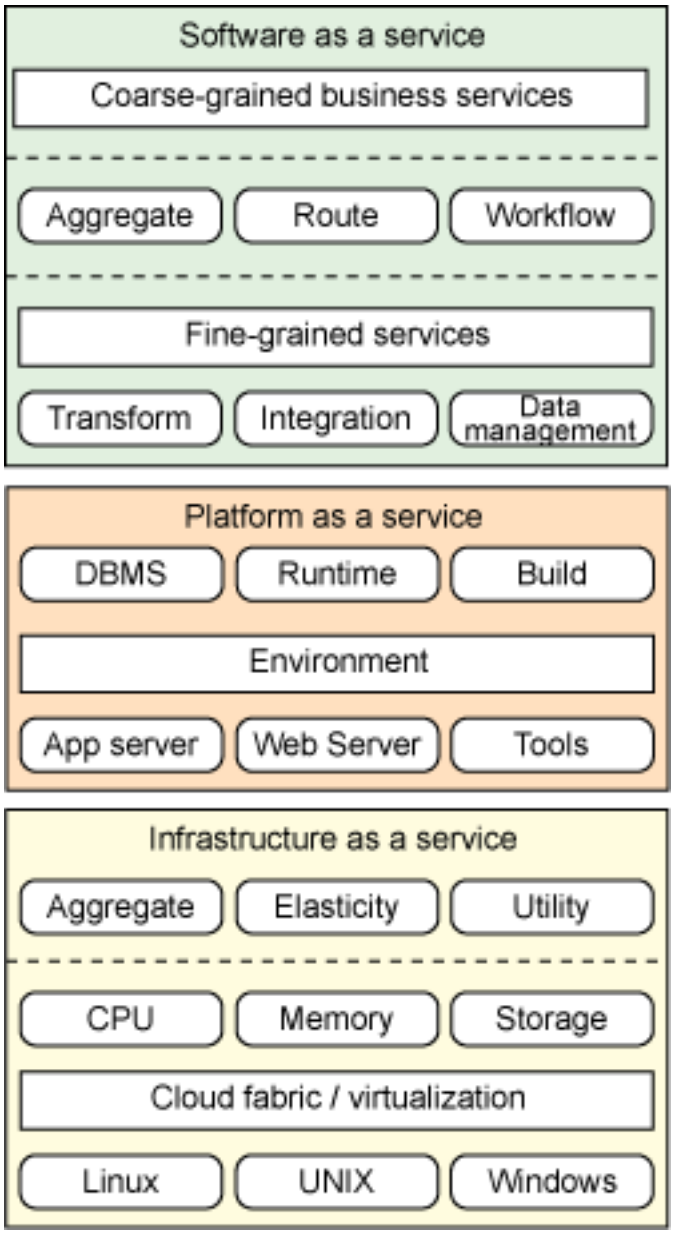
We thought it would be a novel idea to add even more power to the concept of container services in JEE by adding support for some of the powerful ideas of cloud computing — namely parallelism, elasticity, multi-tenancy, and security. This article describes the strategies and patterns to extend JEE containers and applications with cloud computing characteristics. It includes:

- An outline of each cloud characteristic we integrated.
- A layout of the existing characteristics of JEE applications.
- A description of our approach to extend the JEE container to the cloud.
- A design strategy for this type of migration, one that includes the concepts of parallelism, synchronization, storage, elasticity, multi-tenancy, and security.

Cloud characteristics

Figure 1 explains what cloud is and the different cloud deployment models.

Figure 1. A bird's eye view of cloud service models and their components



At the bottom of the cloud stack is the Infrastructure as a Service (IaaS) level. Here the infrastructure has moved to cloud and the cloud now facilitates the deployment of software including business applications. However the IaaS does not have an application-development environment or any testing services. As the figure shows, the top level of abstraction is elasticity, automated deployment, and utility computing.

The Platform as a Service (PaaS) level provides an environment for application software to be created, deployed, and maintained. The PaaS provider has to give the basic life cycle services like build, deploy, testing and building block services like state management, transaction, and security, as well as resource management services through the runtime.

The Software as a Service (SaaS) level provides an environment for the end-user to access an application and use it.

The basic cloud characteristics that an application needs to support are **elasticity** and **multi-tenancy**. Other characteristics, like provisioning and automation, are supported through the deployment features of the application server and do not have much of an impact on the code. Parallelism, distributed storage needs, and security enhancements act as supporting characteristics that need to be addressed to achieve elasticity and multi-tenancy.

Let's look at each in more detail.

Elasticity

Elasticity is the ability to scale up and down the infrastructure based on need. During peak load times, more instances are added to the cluster and when the load comes down, the number of instances comes down. This should be done dynamically. This function is enabled by features of the application server to support dynamic clustering techniques.

Elasticity is not just an application server solution; the application itself should be able to support elasticity. This means the application needs to be designed to handle the resources that it uses to support concurrency. By designing or customizing an application to support elasticity, you imply that you've also implemented parallelism, statelessness, and transaction support in your application.

The design strategy section describes how to implement elasticity that has all the resources support statelessness in execution and parallelism.

Multi-tenancy

Multi-tenancy means your application has the ability for a single application instance to cater to multiple customers; this means that if five customers are using a content management service, then all five customers can use the same application instance with adequate segregation of data and execution parameters. To support multi-tenancy, your application needs to engage distributed storage, parallelism, security, and loose coupling.

There are two approaches to support multi-tenancy:

- A single physical storage for all tenants.
- Multiple physical storage resources for tenants.

Parallelism and transaction support

In the content of this article, *parallelism* is the ability to execute multiple requests in

parallel or to split a large dataset task into multiple subtasks which are executed in parallel. This makes better use of available resources. Employing parallelism has a positive impact on throughput and performance. *Transaction support* ensures reliability by guaranteeing that changes in state of any resource are synchronized. These two concepts sit on opposite ends of a spectrum - if you do more of one, you do less of the other.

The right mixture of parallelism and transaction support is essential to balance these opposing characteristics. The strategies section introduces four strategies, two each for parallelism and transaction support:

- A synchronous and asynchronous approach for parallelism.
- A thread-completion and a data-arrival synchronization approach for transaction support.

The migration strategy described follows non-functional approaches to parallelism, but there are some that require functional changes. Like the Google framework MapReduce; MR describes a way of implementing parallelism using the `Map` function which splits a large data into multiple key-value pairs. (See Resources for articles on MapReduce and the cloud.)

Loose coupling and statelessness

Loose coupling ensures that every call to a service is made through a standard interface; this enables the called component and caller component to be changed without one impacting the other. Loose coupling is introduced by a proxy which invokes the call. *Statelessness* is a property of loose coupling in which every call to a service does not depend on the previous call. It is achieved by managing state changes in a persistent storage.

Both of these are complimentary characteristics that make system calls more independent of dependencies.

Distributed storage

Distributed storage is a means to persist data so that the location of the data is not important. It also means that there are different places where the same data can be stored. This characteristic improves elasticity and statelessness, but can negatively impact transaction support, so it will require a balancing act.

Four strategies for distributed storage include:

- Replicated nodes: Data is available at different nodes and is replicated immediately to other nodes.
- Replication on-demand: Triggers are defined that cause data replication

manually or automatically.

- One-way replication with failover: The master-to-child node plan; during a master node fail, replication duties are assigned to a specific child.
- File system sharing: Used when replication is costly like with file system resources.

Security

Cloud application security impacts certain characteristics strongly: Multi-tenancy, parallelism, and loose coupling introduce additional security needs. And if your application is deployed as a hybrid (for example, a cloud component and a local system component), you need to ensure a cross-domain, single sign-on capability which carries additional security implications.

There are also security issues with distributed storage, parallelism, and transport.

Now that you are familiar with cloud application characteristics, let's look at a Java EE container structure.

Java EE container application characteristics

Traditional JEE applications depend on container services and use:

- Sticky sessions for connection state management
- RDBMS either directly through SQLs or stored procedures indirectly using ORM
- JMS objects

They may also use message-driven Beans and Session Beans and web services implemented using the framework provided by the container. The newly built applications might use asynchronous processing, as well as caching services or JavaMail services.

Let's examine some attributes and functions of JEE container applications in detail.

Data and operation

Every bit of programming logic can be abstracted into a `data- (or memory-) related part` and an `operation- (or execution-) related part` which interacts with each other so that the operation works on data and data is used by operation. The entire JEE package, container and application, can be abstracted in the same manner.

Container

The quality of data aspect is measured by the ability to ensure reliability of data accessed, availability of data accessed, being able to allow concurrency as well as security of the data in storage. The quality of operation aspect is measured by being able to ensure a listener's ability to listen to arrival of data, ability to invoke a remote call as well as access control and transport security.

Table 1. Providing quality for the data and operation aspect of a JEE application

	Quality attribute	Implementation attribute	Implementation
Data	Reliability	Transaction	Transactions provide synchronized access to the data.
	Availability	Persistence	The type of persistence determine availability of data.
	Concurrency	State management	The state management mechanism ensures how many concurrent requests can be processed.
	Security	Security	The encryption in storage and transit.
Operation	Asynchronous communication	Listener	The trigger for asynchronous calls.
	Synchronous communication	Remote invocation	The synchronous call outside the current process.
	Security	Security	The access control check as well as transport security.

The responsibility of container is two-fold:

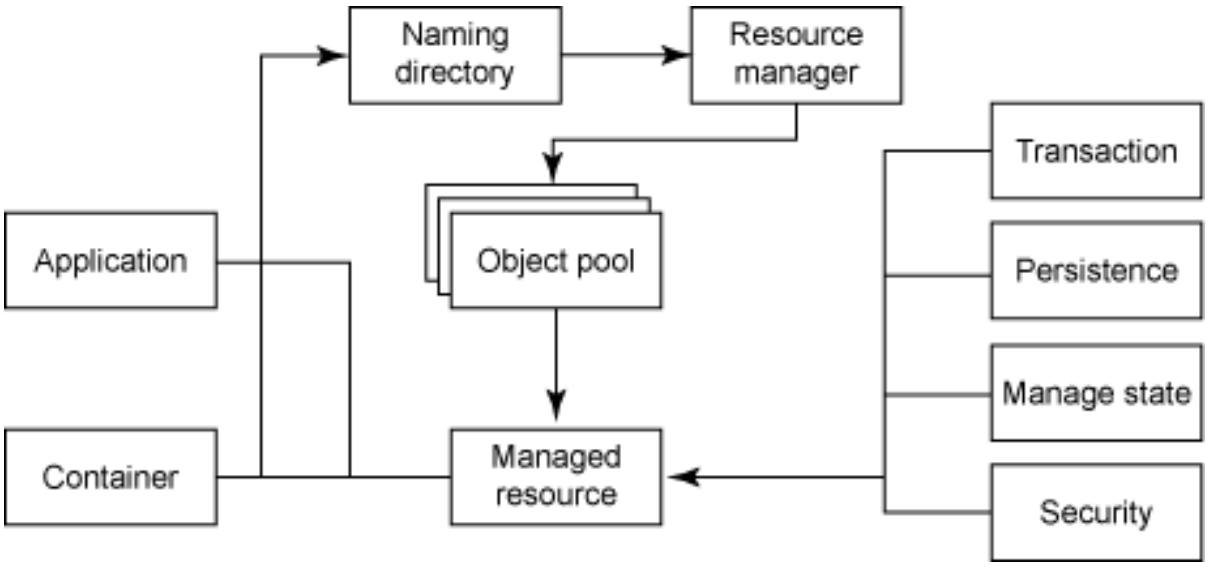
1. To have a mechanism to ensure that the quality attributes of data and operation are maintained.
2. To control the usage of system resources like heap memory, number of execution threads, etc.

This leads to two distinct patterns you should be concerned with — the managed resource pattern and the managed task pattern.

Managed resource pattern

A managed resource provides a data-related service and it implements session management, transaction, persistence, and security. The caller uses the naming directory to locate the resource manager. The resource manager uses the object pool provided by the container to manage system resources. A typical managed resource has the pattern you see in Figure 2.

Figure 2. The managed resource pattern

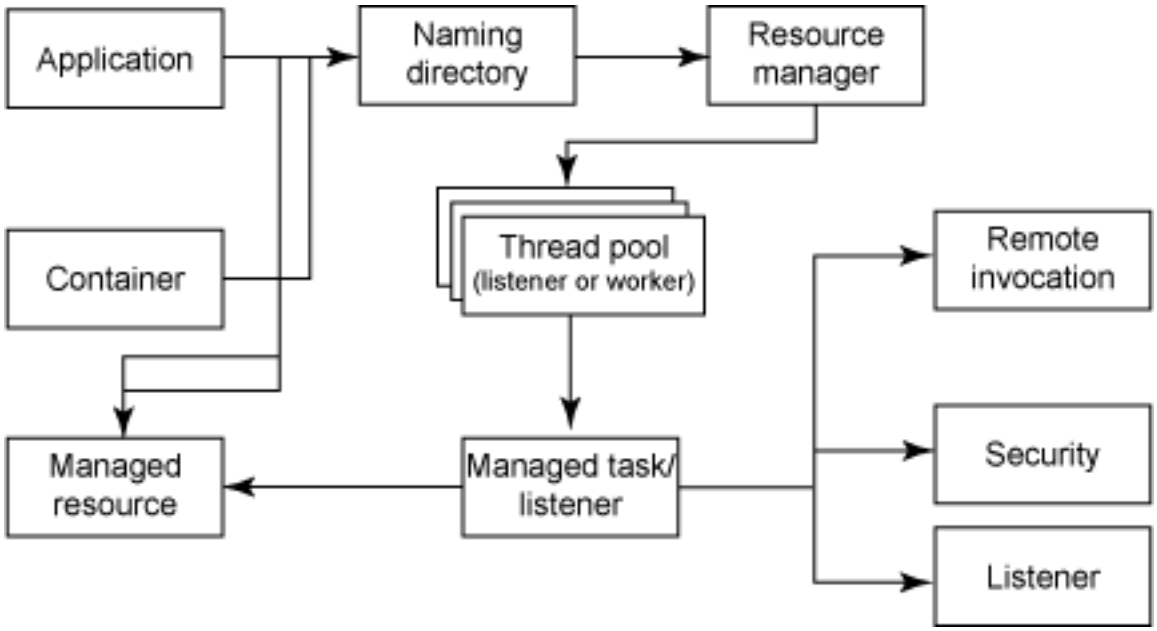


The container or application can get a handle on the resource manager through JNDI. The resource manager implements the object pool and it gets the managed resource that implements persistence, security state management, and transaction.

Managed task pattern

A managed task provides operation-related services that implements remote invocation, listener, and security and it uses the thread pool and naming directory services provided by the container. In addition, a managed task most likely encapsulates one or more of the managed resources that it works on. The managed listener is triggered by the container based on data arrival — the data can be in the form of time, message, or request. It also can be triggered by the application as well.

Figure 3. The managed task pattern



Every service that container provides can be decomposed into one of the patterns or into a combination of the two patterns. For example, Java Message Service (JMS) has a managed resource pattern for JMS Destinations and a managed task pattern for JMS MessageListener. Similarly JDBC Connection is a managed resource pattern.

Now that we have covered how the JEE container application functions, let's look at how to extend a container application to the cloud.

Extending containers: The basic approach

The approach for extending container to cloud is to:

1. Decompose the cloud characteristics into the implementation attributes and then
2. Enhance the managed resource pattern and managed task pattern with the implementation attribute-related changes.

The strategy section shows how the managed resource pattern is extended to the cloud resource pattern and the managed task pattern is extended to the cloud task pattern.

The managed resource pattern employs the following extensions to create the cloud resource pattern (see Figure 4):

- CloudResource
- Isolator
- Replicator
- LockManager
- LockDataResource
- StateDataResource

Similarly the managed task pattern is extended with Proxy and StateManager to create the cloud task pattern (see Figure 5).

Let's discuss some of these components.

Cloud resource pattern

The cloud resource pattern includes the list of extensions just mentioned. Here is a description of each component and their interactions with each other.

CloudResource
The CloudResource extends the managed resource to include distributed transactions and state persistence logic, if needed.

StateDataResource
The StateDataResource is an instance of CloudResource that represents a state change for the given cloud resource. The state persistence logic itself is executed in a stateless manner.

Isolator
The Isolator uses a control field in the input to identify the customer tenant and applies the relevant security and partition logic to store in the correct place. The Isolator ensures that the application code is not cluttered with the multi-tenant storage strategies and ensures that right multi-tenant strategy is applied. The Isolator in itself is a collection of CloudResources.

Replicator
The Replicator is used only if replicated nodes and replication on demand storage strategies are used. The Replicator ensures that the data is persisted in all the replicated nodes as a single distributed transaction. The difference between Isolator and Replicator is that Isolator ensures data goes into correct storage based on the tenant and Replicator ensures data goes into all the storages replicated for same tenant.

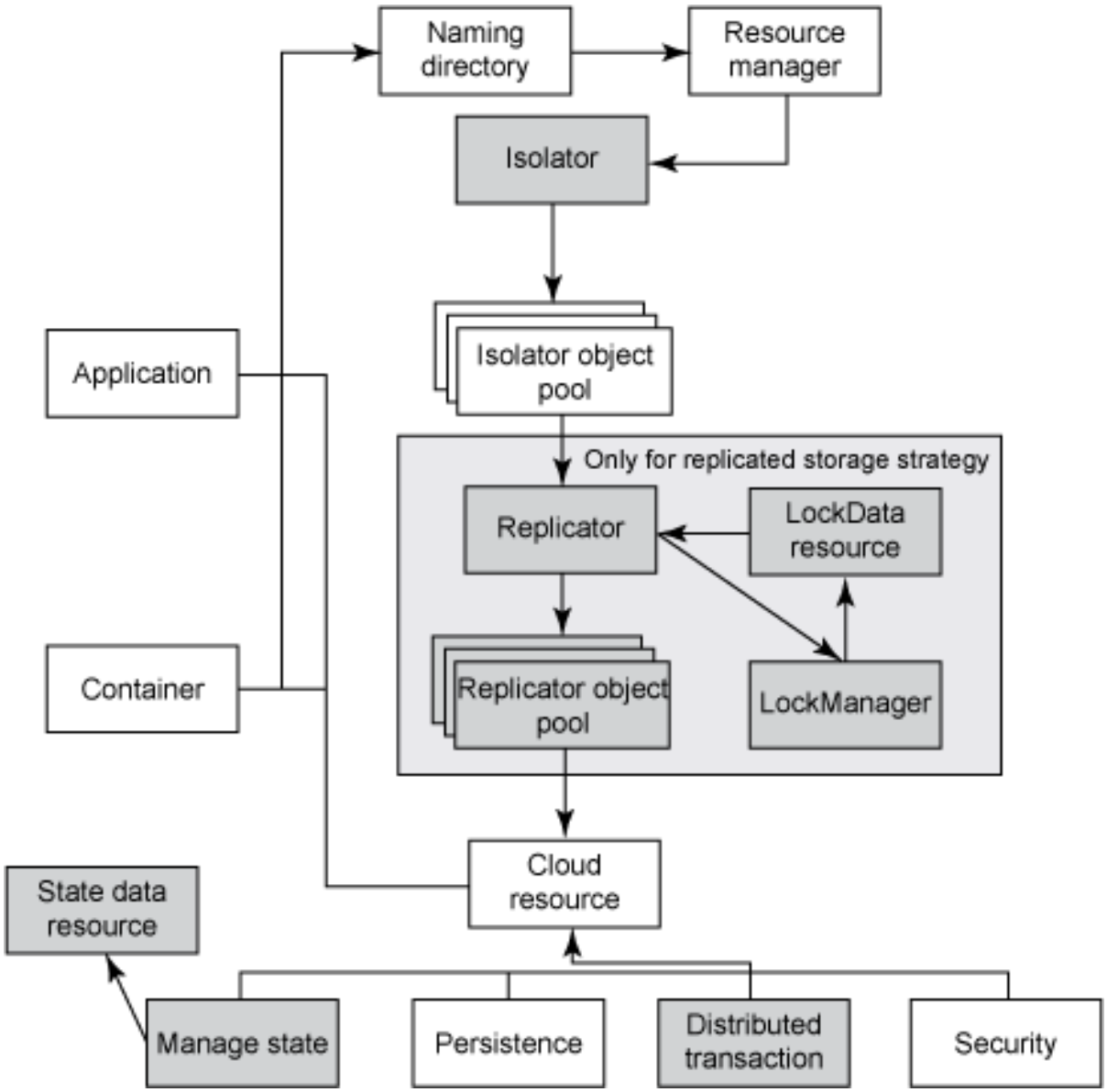
LockManager and LockDataResource
The functionality of LockManager is to lock a particular data for a thread in a process

across all Replicators. The LockManager ensures the same view of status across all replicated nodes. It means that if data is locked for a thread in a server process in server 1, the server 2 process will see the status as locked even if it looks at a replica of storage. This feature is needed only for replicated nodes and replication on demand storage strategies.

The overall changes to the pattern can be summarized as follows (Figure 4):

- The resource manager now provider Isolators which in turn provides a CloudResource directly or a Replicator depending on storage strategy.
- The cloud resource now supports distributed transactions and state management now handles state persistence as well.

Figure 4. The cloud resource pattern now supports distributed transactions



Cloud task pattern

The cloud task pattern extends the managed task pattern with the Proxy and StateManager extensions. The Proxy determines the parallelism strategy and instructs the StateManager to control the persistence of state for the execution.

Proxy

The Proxy is the wrapper around the managed task with pre-process and post-process logic. The pre-process logic includes the message security, followed by formatting the input based on protocol and performing the task. Subsequent to the task execution, the post-process logic decides what to do with the output.

StateManager

The stateless execution of a task is to ensure that input to the task is the initial state and all final state related information is present in the output. Therefore, the StateManager takes care of input and output and moving them as a CloudResource.

Figure 5. The cloud task pattern's StateManager moves I/O as a CloudResource

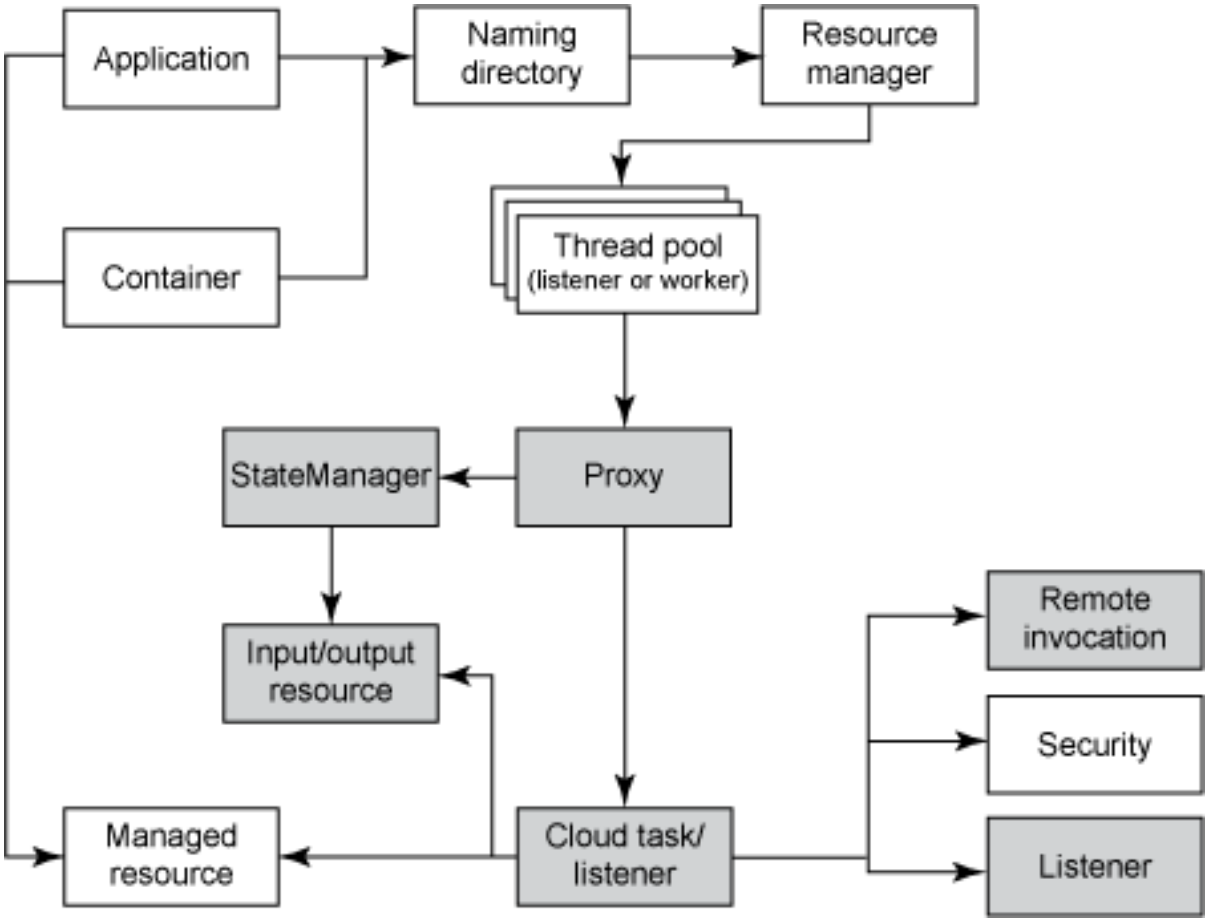


Table 2 shows the details of how each cloud characteristics and its corresponding design strategy impacts which JEE implementation attribute and what patterns are referenced.

Table 2. Cloud characteristics and their impact on design and implementation strategy

Cloud characteristics	Design strategy	Implementation attribute	Pattern	Pattern extensions
Statelessness	Statelessness through state persistence	Listener, remote invocator	Cloud task	StateManager
Statelessness	Statelessness through state persistence	State management	Cloud resource	StateDataResource

Distributed storage	Replicated nodes, Replication on demand	Persistence	Cloud resource	Replicator, LockManager, LockDataResource
Distributed storage	Replicated nodes, Replication on demand	Transaction	Cloud resource	CloudResource
Parallelism and synchronization	All the strategies	Listener, remote invocator	Cloud task	Proxy
Loose coupling	All the strategies	Listener, remote invocator	Cloud task	Proxy
Multi-tenancy	All the strategies	Persistence	Cloud resource	Isolator
Security	Encryption	Persistence	Cloud resource	Isolator
Security	Encryption	Listener, remote invocator	Cloud task	Proxy

Extending containers: Approach for common container services

Modify the existing container services to match the cloud resource and cloud task patterns and attach them to the application in as non-intrusive manner as possible. In a nutshell, we converted all services to cloud resource pattern; when the application interacts with the cloud resource pattern, it converts that pattern to the cloud task pattern and is ready for the cloud. The following list shows the service, the original method, and the approach we used.

- *Service:* **JDBC Database Connections**
  Legacy method: Managed resources. *Approach:* Go for the higher versions that support distributed transactions (two-phase commit), shareable connection that support thread pool and stateless invocation. Based on the higher versions that exist, the remaining functionality can be provided using a cloud resource pattern.
- *Service:* **JMS objects**
  Legacy method: The JMS Senders and Receivers are tasks and JMS messages and destinations are objects.
  Approach: Same approach as for JDBC Database Connections. The configuration can be changed to ensure that the JMS Server is also present on all the nodes where JMS client is also present to help elasticity.
- *Service:* **Cache objects**
  Legacy method: Currently support in-memory or distributed cache services.
  Approach: All caches need to be converted to a distributed cache to take

advantage of effective sharing. The cache services can be optionally wrapped by a cloud resource adapter.

- *Service:* **Session**
  Legacy method: Most of the applications use sticky sessions.
  Approach: The code can be changed in a non-intrusive manner by having a filter for all the requests and let the filter create a custom HttpServletRequestWrapper which can override `getSession()` to give it as a cloud resource. Eliminate sticky session as well.
- *Service:* **Persistence strategies**
  Legacy method: The ORM-based container-managed persistence will be beneficial.
  Approach: The Object-Relational-Mapping-based container managed persistence does not clutter the application code with the relational nature of storage. This enables ease of changing the persistence layer to a non-relational DB as well. Hibernate Shards allows for distributed storage as well and works like the Replicator in the cloud resource pattern. If the application uses a DAO (data access object) with dynamic SQLs and stored procedures, the value object that is passed to DAO can be declared as a cloud resource using annotations.
- *Service:* **Variable**
  Legacy method: The treatment of variables.
  Approach: All public variables and static variables, all file input/outputs, and all log writing are modified to have to use as a cloud resource.
- *Service:* **Method calls**
  Legacy method: The treatment of method calls.
  Approach: All relevant method calls are converted to a cloud task.

Impact on design strategies

Finally, let's look at how enabling cloud characteristics in your application can affect design strategies. We're going to examine the following:

- Synchronous and asynchronous parallelism.
- Data-arrival and thread-completion synchronization.
- Replicated node, replication on demand, one-way replication with failover, and file system sharing on the storage side.
- Loose coupling and statelessness.
- Elasticity.
- Single- and separate-storage multi-tenancy.

- Security.

Parallelism

As mentioned before, parallelism implies the parallel-processing capability of a task where the task is split to different subtasks with consolidation at the end. Two strategies for parallelism are offered, synchronous and asynchronous.

Synchronous

Here the caller waits for the execution to complete before proceeding to the next task. Each task is triggered using a service call. The task waits for the caller to return. Parallelism is introduced by running the main thread scheduling child threads for each task so that the tasks execute concurrently while each thread execute the task synchronously.

The synchronous strategy is best suited for data aggregation. When the application needs information from different sources, data from each source can be got synchronously but each source can be fetched concurrently.

Asynchronous

Here the caller uses messaging to invoke a task asynchronously. When the task is completed, the output of the task is kept in a persistent storage area for the next task to pick it up.

This is more suited for orchestration tasks. Each orchestration task is invoked asynchronously and each of them calls data aggregation tasks using synchronous strategy.

Synchronization strategies

Synchronization strategies ensure transaction support and therefore the reliability of the task. It ensures the relevant tasks and relevant data is made available before the next task is processed. There are two synchronization strategies, data-arrival based and thread-completion based synchronization.

Data-arrival based

Data-arrival synchronization implies that a task is triggered based on a particular data arriving at a particular location. This is especially important when asynchronous invocation is used where the calling task has to look for the output of the called task appearing at a particular location to continue further.

A thread polls the data area and checks for the data's arrival. It sends a message for the task to be triggered as soon as data arrives. The MessageListener is one implementation of data arrival synchronization.

Thread-completion based

In thread-completion synchronization, a resource monitor looks at concurrent threads and synchronizes the execution of the tasks while accessing that resource. The same strategy can be applied for a resource that has to be shared across instances using a combination of the data-arrival strategy as well as the thread-completion strategy.

This strategy is generally applied for the threads within a single JVM. For external resources, this approach is an extension of data-arrival synchronization where the monitor status is the data and the listener brings in the data to the thread executing in current JVM.

Storage

There are different types of storage; databases, file systems, in-memory data, persistence devices like native directories are but a few examples. The critical needs for a storage strategy is to ensure that JEE application has a way to create a true network object, an object whose state change is reflected in all JVMs in the cluster and have a way to migrate to a distributed data source when dynamic RDBMS SQL does not work. In addition, the storage strategy forms an important component for parallelism and multi-tenancy.

Replicated nodes

The replicated nodes strategy implies that the same data is available at different nodes and is replicated immediately to other nodes. The warm and hot replication strategies are applicable to databases. The other storage areas need to use the strategy applied in two-phase commit.

There are two threads for the execution support at a bare minimum. One thread has a Save/Update action which it invokes synchronously. This sends the data via socket connections to other nodes and upon receiving the response, it issues the commit which actually updates the nodes. The listener thread listens for the changes from other nodes and responds to them. The socket connection can be replaced with messaging or http based services.

The replication is best suited for read-only data with fewer updates. The replication is costly. All caching and in-memory data can be redesigned as replicated nodes based storage.

Replication on demand

This is a variation of replication strategy where the regular commits are for the current node and the code can trigger replication at a logical point. The replication can also be trigger on an as-needed basis using the synchronization strategies.

Replication on demand helps in the scenario where each node works with different pieces of data and will be combined at logical steps. A MapReduce-style programming model can use this strategy.

One-way replication with failover

Yet another variation of replication is to assign a master node and all data is replicated from master to child nodes. During failover, one of the children becomes the master node and all replication starts flowing from that node. The application always works with the master node. This strategy works along with file system sharing strategy to design for high availability.

File system sharing

This is commonly used for file system based resources where replication is costly. Here the file system itself moves around different nodes to do load balancing. This does not provide a foolproof availability option, but is sufficiently close to that. The relational database file systems can be handled in this manner.

Loose coupling and statelessness

Statelessness is achieved by ensuring that all the data needed for the call is available and there is no dependency on the machine that processed the request. Loose coupling ensures that a call is replaceable. An HTTP-based REST or web service call can ensure both.

Elasticity

Elasticity is achieved by ensuring right mix of parallelism support and transactional support and more importantly, that the execution is stateless and therefore repeatable. The state of each execution is persisted separately. The same instance of an object can have multiple threads under execution; each thread has a separate area for the object. This reduces the need of multiple objects in the pool. Some of the database providers already provide a shareable connection to take care of this feature. This means about 2 to 3 connections are sufficient to handle 20 to 30 concurrent shared read transactions on an average.

The smaller the transaction duration, the better it is for elasticity. It makes sense to split a long running task to a number of repeatable stateless tasks to reduce transaction size.

Multi-tenancy

Multi-tenancy implies one instance of the application can actually service multiple clients with adequate separation of data. There are different design considerations for multi-tenancy, but the critical one which can be tackled without code refactoring is addressing single vs. separate storage for different tenants.

Single storage for multiple tenants

This means there is only one storage resource for all customers. This configuration is more maintainable and more scalable since there is only a logical segregation and therefore each individual data element may be encrypted by a separate key. The schema and partition provide logical separation of data for databases and

namespace provide logical separation of data for XML.

Separate, individual storage for each tenant
Each customer gets their own storage. Here there is physical data segregation. This model is suited for applications where policy dictates that data is sensitive in nature. This separate storage should be accessed using a routing service to keep the complexity from crowding the programming logic.

Security

Additional security needs arise due to distributed storage and multi-tenancy. The security strategy needs to address storage, messaging, and transport security. In addition, a hybrid cloud or virtual private cloud implementation would require a cross-domain single sign-on option, therefore federation will play a role as well.

The additional security needs are an offshoot of application-specific cloud characteristics, therefore this can be implemented without impacting the rest of the functionality and code.

In conclusion

The methodology we have outlined provides design strategies and patterns to help you enable cloud characteristics in container services and to help you migrate applications to the cloud. The cloud resource pattern and cloud task pattern are designed to be reused and we make no assumptions on the strategy you will choose.

We've examined the characteristics that make cloud applications unique, as well as corresponding characteristics of Java EE containers and applications. we have adapted the managed resource and managed task patterns into cloud resource and cloud task patterns to show you how to extend container services to be used by cloud applications.

Also, we have pointed out the impact of integrating JEE containers to cloud application on your cloud application design strategies.

Finally, we would like to note that the JEE Connector Architecture (JCA) is one of the options to implement the extensions to managed resource and managed task patterns. JEE Connector Architecture provides the framework to define a resource with work, life cycle, connection, and transaction management, as well as to enhance security architecture, inbound communications, and the messaging inflow.

Resources

Learn

- Java technology at developerWorks has multitudes of resources on using containers in Java EE.
- For a more functional approach to application parallelism, see
  - Discover how MapReduce and cloud computing are ideal for dealing with lots of data in Solve cloud-related Big Data problems with MapReduce. November 2010
  - See how MapReduce and virtualization improves node performance in Using MapReduce and load balancing on the cloud. July 2010
- Following are references to concepts used in this article:
  - The Five Characteristics of Cloud Computing has a good section on elasticity.
  - Storage Multi-Tenancy for Cloud Computing is a good reference on multi-tenancy.
  - Best practices for getting Java to work for multicore processors details parallelism and transaction support.
- In the developerWorks cloud developer resources, discover and share knowledge and experience of application and services developers building their projects for cloud deployment.
- The next steps: Find out how to access IBM SmartCloud Enterprise.

Get products and technologies

- See the product images available on the IBM Smart Business Development and Test on the IBM Cloud.

Discuss

- Join a cloud computing group on developerWorks.
- Read all the great cloud blogs on developerWorks.
- Join the developerWorks community, a professional network and unified set of community tools for connecting, sharing, and collaborating.

About the authors

Jayakrishnan Ramdas

Jayakrishnan Ramdas has more than 15 years of IT industry experience working for Infosys LTD, a well-known software services vendor. With more than 6 years of architecting experience, Ramdas has completed TOGAF (an industry-standard architecture framework) and also various IBM certification programs.

J. Srinivas

J. Srinivas has 16 years of industry experience and is currently heading the technology focus group for Infosys LTD's Banking and Capital Markets unit for Europe. He has worked in the project management and architecture streams and is passionate about process, technology, and building motivated teams. He is a core member of the Infosys agile process team and is a cloud computing evangelist with Infosys.